

Web Application Development

Lecture 13: JWT Authentication

SWE 381 – Web Application Development

Dr. Ohoud Alharbi Dr. Achraf Gazdar

Department of Software Engineering
College of Computer and Information Sciences
King Saud University

Term 2 – 2025/2026

Learning Objectives

By the end of this lecture, you will be able to:

- Distinguish between **Authentication (AuthN)** and **Authorization (AuthZ)**
- Hash passwords securely with **bcrypt.hash()** and verify with **bcrypt.compare()**
- Explain the **three-part JWT structure**: header, payload, signature
- Sign tokens with **jwt.sign()** and verify them with **jwt.verify()**
- Implement **register** and **login** routes with bcrypt and JWT
- Build a **protect middleware** that reads and verifies Bearer tokens
- Test the full authentication flow in **Postman** with Bearer tokens

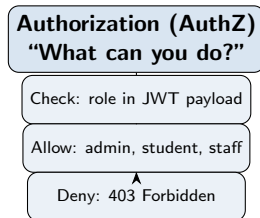
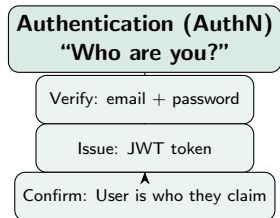
Agenda

- 1 AuthN vs AuthZ
- 2 bcrypt.hash() & compare()
- 3 JWT Structure
- 4 jwt.sign() & verify()
- 5 Register & Login Routes
- 6 protect Middleware
- 7 Bearer Token in Postman
- 8 Recap

Outline

- 1 AuthN vs AuthZ
- 2 bcrypt.hash() & compare()
- 3 JWT Structure
- 4 jwt.sign() & verify()
- 5 Register & Login Routes
- 6 protect Middleware
- 7 Bearer Token in Postman
- 8 Recap

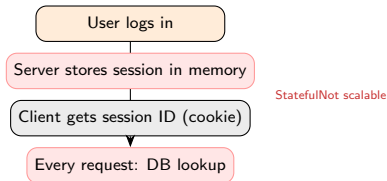
Authentication vs Authorization



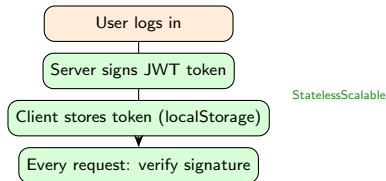
	Authentication (AuthN)	Authorization (AuthZ)
Question	Who are you?	What are you allowed to do?
Happens	At login	On every protected request
Tool	Password check + JWT issue	JWT decode + role check
Failure	401 Unauthorized	403 Forbidden

Session-Based vs Token-Based Auth

Session-based (traditional):



JWT token-based (modern):



W3Schools – Node.js API Authentication

JSON Web Tokens (JWT) provide a **stateless authentication mechanism** that's compact and self-contained. Unlike session-based authentication, token-based authentication **doesn't require a server to store session data**. This makes it ideal for stateless API architecture and microservices.

Outline

- 1 AuthN vs AuthZ
- 2 `bcrypt.hash()` & `compare()`
- 3 JWT Structure
- 4 `jwt.sign()` & `verify()`
- 5 Register & Login Routes
- 6 protect Middleware
- 7 Bearer Token in Postman
- 8 Recap

Why Hash Passwords?

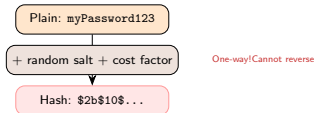
⚠ Never store plain-text passwords

If your database is ever breached, plain-text passwords expose every user's credentials. **bcrypt** is a one-way hashing algorithm – you can verify but never reverse it.

```
1 // Install: npm install bcrypt
2
3 const bcrypt = require('bcrypt');
4 const saltRounds = 10;
5
6 // --- HASHING a password ---
7 const plainPassword = 'myPassword123';
8
9 const hashedPassword =
10   await bcrypt.hash(plainPassword, saltRounds);
11
12 console.log(hashedPassword);
13 // $2b$10$9...very long random string...
14 // Always different even for same password!
15 // saltRounds = 10 is the recommended default
16
17 // --- COMPARING password to hash ---
18 const isMatch = await bcrypt.compare(
19   plainPassword,
20   hashedPassword
21 );
22 console.log(isMatch); // true
23
24 const isWrong = await bcrypt.compare(
25   'wrongPassword',
26   hashedPassword
27 );
28 console.log(isWrong); // false
```

Terminal

```
$2b$10$N9qo8uL0ick...abc
true
false
```



💡 Salt rounds

`saltRounds = 10` is the standard. Higher = slower but more secure. 10 takes ~100ms, 12 takes ~400ms.

bcrypt in Practice – User Registration

W3Schools – API Authentication (bcrypt)

Hash password with `bcrypt.hash(password, saltRounds)` before storing. **Never** store the plain-text password in the database.

```
const bcrypt = require('bcrypt');
const express = require('express');
const app = express();
app.use(express.json());

const users = []; // in-memory store

// POST /register
app.post('/register', async (req, res) => {
  try {
    const { username, password } = req.body;

    // 1. Check duplicate username
    if (users.find(
      u => u.username === username)) {
      return res.status(400).json(
        { message: 'Username taken' });
    }

    // 2. Hash password (NEVER store plain!)
    const saltRounds = 10;
```

```
const hashedPassword =
  await bcrypt.hash(
    password, saltRounds);

// 3. Store with hashed password
const newUser = {
  id: users.length + 1,
  username,
  password: hashedPassword // hashed
};
users.push(newUser);

res.status(201).json({
  message:
    'User registered successfully',
  userId: newUser.id
});

} catch (error) {
  res.status(500).json(
    { message: 'Error registering' });
}
});
```

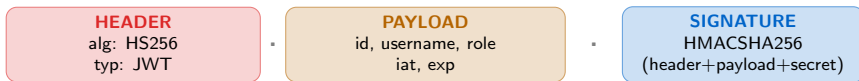
Outline

- 1 AuthN vs AuthZ
- 2 bcrypt.hash() & compare()
- 3 JWT Structure**
- 4 jwt.sign() & verify()
- 5 Register & Login Routes
- 6 protect Middleware
- 7 Bearer Token in Postman
- 8 Recap

JWT Structure – 3 Parts

W3Schools – JWT Authentication

JWT is made up of three parts: **header**, **payload**, and **signature**, all **separated by dots**. JWT uses base64 to encode each component. The header defines the token type and algorithm; the payload contains user data such as ID or role; the signature ensures the integrity of the token.



A real JWT token looks like:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  
eyJpZCI6MSwidXNlcm5hbWUiOiJBaG1lZCIsInJvbGUiOiJzdHVkZW50In0.  
SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

JWT Decoded – Header and Payload

Header (decoded):

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Algorithm and token type.

HS256 = HMAC-SHA256.

Payload (decoded):

```
{
  "id": 1,
  "username": "Ahmed",
  "role": "student",
  "iat": 1706000000,
  "exp": 1706003600
}
```

iat = issued at (Unix).

exp = expires at.

Signature:

```
HMACSHA256(
  base64(header) +
  "." +
  base64(payload),
  JWT_SECRET
)
```

Tamper-proof.

Verified with secret.



Header and payload are NOT encrypted

Header and payload are **only base64-encoded** – anyone can decode them. **Never** put passwords or sensitive data in the JWT payload. The **signature** is what makes it secure.



jwt.io for debugging

Visit **jwt.io** to paste any JWT and see its decoded header and payload. Very useful for debugging during development.

Outline

- 1 AuthN vs AuthZ
- 2 bcrypt.hash() & compare()
- 3 JWT Structure
- 4 **jwt.sign() & verify()**
- 5 Register & Login Routes
- 6 protect Middleware
- 7 Bearer Token in Postman
- 8 Recap

jwt.sign() – Creating a Token

W3Schools – Node.js API Authentication

`jwt.sign(payload, secret, options)` creates a JWT. The payload contains the user data, the secret is your private key (from `.env`), and options include expiry time.

```
// Install: npm install jsonwebtoken

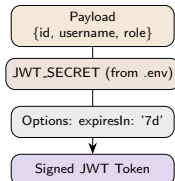
const jwt = require('jsonwebtoken');

// ---- jwt.sign() ----
// Arguments: payload, secret, options
const token = jwt.sign(
  // 1. Payload -- user data (no passwords!)
  {
    id:      user.id,
    username: user.username,
    role:    user.role
  },
  // 2. Secret -- from .env
  process.env.JWT_SECRET,
  // 3. Options
  { expiresIn: '7d' } // expires in 7 days
);

console.log(token);
// eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

// Common expiresIn values:
// '1h'    = 1 hour
// '24h'   = 24 hours
// '7d'    = 7 days
// '30d'   = 30 days
```

```
# .env
JWT_SECRET=swe381-super-secret-key-2025
JWT_EXPIRE=7d
```



jwt.verify() – Validating a Token

W3Schools – JWT Middleware

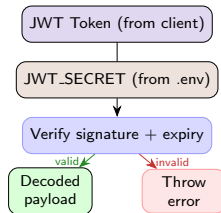
The Authorization header is commonly used to send authentication tokens. `jwt.verify(token, secret)` decodes and verifies the token's signature and expiry. If valid, it returns the decoded payload.

```
const jwt = require('jsonwebtoken');

// ---- jwt.verify() ----
// Called with: token + same secret used to sign

try {
  const decoded = jwt.verify(
    token,
    process.env.JWT_SECRET
  );
  // decoded contains the payload:
  // { id: 1, username: 'Ahmed', role: 'student',
  //   iat: 1706000000, exp: 1706604800 }

  console.log(decoded.id);      // 1
  console.log(decoded.username); // 'Ahmed'
  console.log(decoded.role);    // 'student'
} catch (err) {
  // Token is invalid or expired
  // err.name = 'JsonWebTokenError'
  // err.name = 'TokenExpiredError'
  console.error(err.message);
  // -> "invalid token" or "jwt expired"
}
```



Decoded payload

```
{ "id": 1, "username": "Ahmed",
  "role": "student", "iat": ..., "exp": ... }
```

Outline

- 1 AuthN vs AuthZ
- 2 bcrypt.hash() & compare()
- 3 JWT Structure
- 4 jwt.sign() & verify()
- 5 Register & Login Routes**
- 6 protect Middleware
- 7 Bearer Token in Postman
- 8 Recap

Register Route – Full Implementation

```
// routes/authRoutes.js
const express = require('express');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const router = express.Router();

const users = []; // Replace with DB model

// POST /api/auth/register
router.post('/register', async (req, res) => {
  try {
    const { name, email, password } = req.body;

    // 1. Validate input
    if (!name || !email || !password) {
      return res.status(400).json(
        { error: 'All fields are required' });
    }
    if (password.length < 8) {
      return res.status(400).json(
        { error: 'Password must be >= 8 chars' });
    }
  }
}
```

```
// 2. Check for duplicate email
const exists = users.find(
  u => u.email === email);
if (exists) {
  return res.status(400).json(
    { error: 'Email already registered' });
}

// 3. Hash password with bcrypt
const hashedPassword =
  await bcrypt.hash(password, 10);

// 4. Save user (hashed -- safe to store)
const newUser = { id: users.length + 1,
  name, email, password: hashedPassword };
users.push(newUser);

// 5. Return success (NO token on register)
res.status(201).json({
  message: 'User registered successfully',
  userId: newUser.id
});
} catch (err) {
  res.status(500).json(
    { error: 'Registration failed' });
}
});
```

Login Route – Full Implementation

```
// POST /api/auth/login
router.post('/login', async (req, res) => {
  try {
    const { email, password } = req.body;

    // 1. Find user by email
    const user = users.find(
      u => u.email === email);
    if (!user) {
      // Same message -- prevent user enumeration
      return res.status(401).json(
        { error: 'Invalid email or password' });
    }

    // 2. Compare password with stored hash
    const passwordMatch = await bcrypt.compare(
      password, user.password);
    if (!passwordMatch) {
      return res.status(401).json(
        { error: 'Invalid email or password' });
    }
  }
}
```

```
// 3. Build JWT payload (no sensitive fields!)
const payload = {
  id: user.id,
  name: user.name,
  email: user.email
};

// 4. Sign the token
const token = jwt.sign(
  payload,
  process.env.JWT_SECRET,
  { expiresIn:
    process.env.JWT_EXPIRE || '7d' }
);

// 5. Return token to client
res.status(200).json({
  message: 'Login successful',
  token
});
} catch (err) {
  res.status(500).json(
    { error: 'Login failed' });
}
});

module.exports = router;
```

Register and Login – Postman Flow

Step 1: POST /api/auth/register

```
// Request body:
{
  "name": "Ahmed Al-Farsi",
  "email": "ahmed@ksu.edu.sa",
  "password": "securePass123"
}
```

201 Created

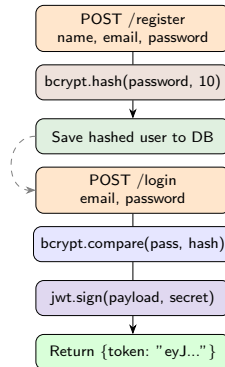
```
{
  "message": "User registered successfully",
  "userId": 1
}
```

Step 2: POST /api/auth/login

```
{
  "email": "ahmed@ksu.edu.sa",
  "password": "securePass123"
}
```

200 OK

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJI..."
}
```



Outline

- 1 AuthN vs AuthZ
- 2 bcrypt.hash() & compare()
- 3 JWT Structure
- 4 jwt.sign() & verify()
- 5 Register & Login Routes
- 6 protect Middleware**
- 7 Bearer Token in Postman
- 8 Recap

The protect Middleware

W3Schools – JWT Verification Middleware

The Authorization header is commonly used to send authentication tokens. The header value follows the format Authorization: Bearer <token>. The middleware **extracts the token, verifies it, and either attaches the user to req or returns 401.**

```
// middleware/protect.js
const jwt = require('jsonwebtoken');

const protect = (req, res, next) => {
  // 1. Read the Authorization header
  const authHeader = req.headers.authorization;

  // 2. Check it exists and starts with "Bearer "
  if (!authHeader ||
    !authHeader.startsWith('Bearer ')) {
    return res.status(401).json(
      { error: 'No token -- access denied' });
  }

  // 3. Extract the token (remove "Bearer ")
  const token = authHeader.split(' ')[1];
  // authHeader = "Bearer eyJhbGciOi..."
  // split(' ') = ["Bearer", "eyJhbGciOi..."]
  // [1]         = "eyJhbGciOi..."
}
```

```
try {
  // 4. Verify signature + expiry
  const decoded = jwt.verify(
    token, process.env.JWT_SECRET);

  // 5. Attach decoded user to req
  // available in all next handlers
  req.user = decoded;
  // { id, name, email, iat, exp }

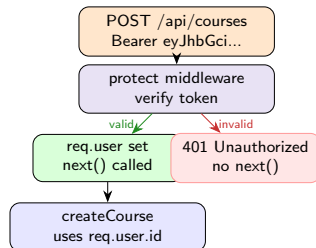
  next(); // 6. Continue to route handler
} catch (err) {
  return res.status(401).json(
    { error: 'Invalid or expired token' });
}

module.exports = protect;
```

Applying protect to Routes

```
1 // routes/courseRoutes.js
2 const express = require('express');
3 const router  = express.Router();
4 const protect = require('../middleware/protect');
5
6 // Public routes -- no auth needed
7 router.get('/', getAllCourses);
8
9 // Protected routes -- must have valid JWT
10 router.post('/', protect, createCourse);
11 router.put('/:id', protect, updateCourse);
12 router.delete('/:id', protect, deleteCourse);
13
14 // In the route handler, req.user is available:
15 function createCourse(req, res) {
16   const { title, dept, credits } = req.body;
17   // req.user was set by protect middleware
18   console.log('Created by:', req.user.name);
19   res.status(201).json({ title, dept, credits,
20     createdBy: req.user.id
21   });
22 }
23
24 module.exports = router;
```

```
1 // index.js -- also protect entire routers
2 app.use('/api/auth', authRoutes); // public
3 app.use('/api/courses', courseRoutes); // mixed
```



💡 req.user is available downstream

After protect calls next(), any subsequent handler can read req.user.id, req.user.name, etc. – just like middleware Lecture 12.

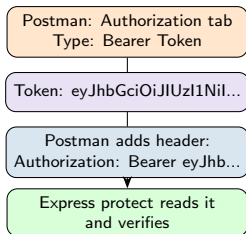
Outline

- 1 AuthN vs AuthZ
- 2 bcrypt.hash() & compare()
- 3 JWT Structure
- 4 jwt.sign() & verify()
- 5 Register & Login Routes
- 6 protect Middleware
- 7 Bearer Token in Postman**
- 8 Recap

Testing with Bearer Token in Postman

Step-by-step Postman workflow:

- 1 Register: POST /api/auth/register with JSON body
- 2 Login: POST /api/auth/login with credentials
- 3 Copy the token from the login response
- 4 Open a protected request (e.g. POST /api/courses)
- 5 Click the **Authorization** tab in Postman
- 6 Select type: **Bearer Token**
- 7 Paste the token in the Token field
- 8 Click **Send**



What Postman sends:

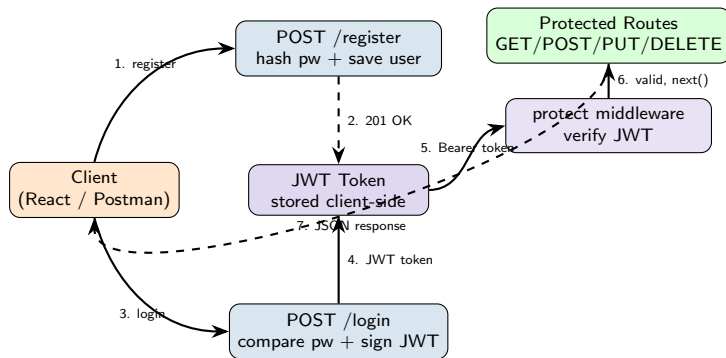
```
POST /api/courses HTTP/1.1
Host: localhost:3000
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSJ1Ym11Ijo1QWh0ZWQiLCJpYXQiOiE3MDYwMDAwMDAsImV4cCI6MTcwNjYwNDgwMH0.Sf1KxwRJSMekKF2QT4fwpMeJf36P0k6yJV
```

```
Body:
{
  "title": "Cloud Computing",
  "dept": "CS",
  "credits": 3
}
```

“Bearer” prefix is required.

The space between Bearer and token is mandatory.

Complete Authentication Flow – End to End



Full Project Setup – index.js

```
// index.js
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const app = express();

// ---- Middleware ----
app.use(cors({
  origin: process.env.FRONTEND_URL }));
app.use(express.json());

// ---- Routes ----
const authRoutes =
  require('./routes/authRoutes'); // public
const courseRoutes =
  require('./routes/courseRoutes'); // mixed
```

```
app.use('/api/auth', authRoutes);
app.use('/api/courses', courseRoutes);

// ---- 404 ----
app.use((req, res) =>
  res.status(404).json(
    { error: 'Route not found' }));

// ---- Global error handler ----
app.use((err, req, res, next) =>
  res.status(err.statusCode || 500)
    .json({ error: err.message }));

app.listen(
  process.env.PORT || 3000,
  () => console.log(
    'Auth API on port ${
      process.env.PORT || 3000}'));
```

```
# .env
PORT=3000
FRONTEND_URL=http://localhost:5173
JWT_SECRET=swe381-super-long-secret-2025
JWT_EXPIRE=7d
```

Folder Structure for Auth

```
swe381-backend/  
  routes/  
    authRoutes.js    <- /register, /login  
    courseRoutes.js  <- /api/courses  
  middleware/  
    protect.js       <- JWT verify guard  
    logger.js        <- request logger  
    errorHandler.js  <- global error  
  index.js           <- entry point  
  package.json  
  .env               <- JWT_SECRET, PORT  
  .gitignore         <- node_modules, .env
```

Auth API endpoints summary:

Method	Route	Auth?
POST	/api/auth/register	No
POST	/api/auth/login	No
GET	/api/courses	No
POST	/api/courses	Yes
PUT	/api/courses/:id	Yes
DELETE	/api/courses/:id	Yes

React frontend usage:

```
1 // After login, save token:  
2 const { token } = await response.json();  
3 localStorage.setItem('token', token);  
4  
5 // On every protected request:  
6 const token =  
7   localStorage.getItem('token');  
8  
9 const res = await fetch('/api/courses', {  
10   method: 'POST',  
11   headers: {  
12     'Content-Type': 'application/json',  
13     // Bearer token in every request:  
14     'Authorization': 'Bearer ${token}'  
15   },  
16   body: JSON.stringify({ title, dept })  
17 });
```



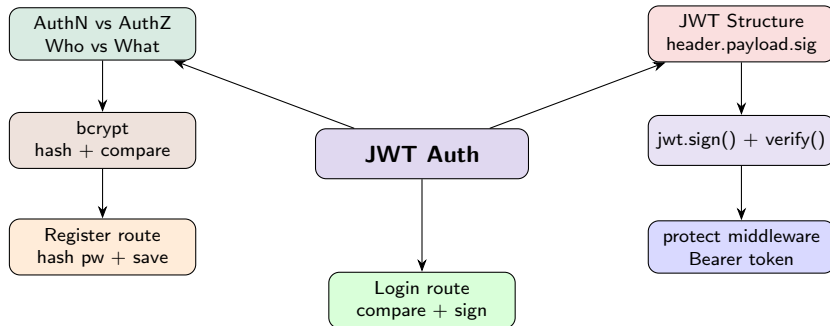
localStorage vs cookies

Store JWT in localStorage for simplicity (SPAs). For production apps, **httpOnly cookies** are more secure against XSS attacks.

Outline

- 1 AuthN vs AuthZ
- 2 bcrypt.hash() & compare()
- 3 JWT Structure
- 4 jwt.sign() & verify()
- 5 Register & Login Routes
- 6 protect Middleware
- 7 Bearer Token in Postman
- 8 Recap

JWT Authentication – Concept Map



Lecture 13 – Key Takeaways

- 1 **AuthN** (authentication) = verify who you are (login); **AuthZ** (authorization) = check what you're allowed to do (roles)
- 2 JWT is **stateless** – the server stores nothing; the token carries all user info and is verified on every request
- 3 **bcrypt.hash(password, 10)** produces a one-way salted hash – safe to store in DB; **bcrypt.compare()** checks a plain password against the hash
- 4 JWT has **3 parts**: header (algorithm), payload (user data), signature (tamper-proof) – separated by dots
- 5 Header and payload are base64-encoded (not encrypted) – **never put passwords** in the payload
- 6 **jwt.sign(payload, secret, {expiresIn})** creates a signed token; **jwt.verify(token, secret)** validates and decodes it
- 7 The **protect middleware** reads Authorization: Bearer <token>, calls `jwt.verify()`, attaches decoded user to `req.user`, then calls `next()`
- 8 In Postman: login to get the token, then use the **Authorization tab** with type **Bearer Token** for protected requests
- 9 Store `JWT_SECRET` in `.env` – use a long random string; never hardcode it

Next Lecture: MongoDB & Mongoose – Connecting the API to a Real Database ▶

Practice Exercises

- 1 Install **bcrypt** and write a standalone script that hashes the string "password123" with 10 salt rounds. Then use `bcrypt.compare()` to verify both the correct and wrong passwords. Print the results.
- 2 Install **jsonwebtoken** and write a script that calls `jwt.sign()` with a payload of `{id:1, name:"Ahmed"}` and expiry `'1h'`. Then call `jwt.verify()` on the result. Decode it at jwt.io to see the three parts.
- 3 Implement the complete **POST /api/auth/register** route (Lecture 13 code). Test with Postman: register the same email twice and confirm the second attempt returns 400.
- 4 Implement the **POST /api/auth/login** route. Test: login with correct credentials (get a token), then with wrong password (401), then with non-existent email (401).
- 5 Build the **protect middleware** in `middleware/protect.js`. Apply it to `POST /api/courses`. Test in Postman: attempt with no token (401), expired token (401), and valid token (201).
- 6 Complete the **full authentication flow** in Postman: register a user, login, copy the token, use it as a Bearer token to create and delete a course. Verify that `req.user` is accessible inside the route handler.
- 7 **Challenge:** Add a **role-based authorization** check. Include `role: "admin"` in the JWT payload for admin users. Create an `isAdmin` middleware that checks `req.user.role === 'admin'` and returns 403 if not. Apply it to the `DELETE` route.